

Nbody: retain state across all timesteps

The newest full target-VM run confirms that `flat_pow` reduces median runtime by 37.53%, from 228.25 to 142.58 ms per 20,000-step unit. The earlier grouped change achieved only 3.40%. The larger improvement comes from eliminating repeated container traffic and short-loop machinery while preserving the force calculation and all timesteps.

Candidate / evidence	Median runtime reduction	Decision
grouped, earlier Ubuntu pair	3.40%	Useful intermediate; below 7%.
flat_pow, latest full Ubuntu pair	37.53% / 1.6009x	Exact default; above 7%.
flat_sqrt, separate timing pair	38.58%	Optional approximate experiment.
New structural controls	Exact verification passed	Explain contributions; no extra speedup claimed.

The reference is the pyperformance five-body gravitational benchmark: Sun, Jupiter, Saturn, Uranus and Neptune. Each step processes ten unordered pairs, updates both bodies' velocities, then advances five positions. A unit has 20,000 steps at $dt = 0.01$: 200,000 pair updates and 100,000 position updates. Sixteen timed units accumulate state through 320,000 steps; they do not restart from the initial orbit.

BODIES maps names to tuples containing mutable position and velocity lists plus mass. SYSTEM and PAIRS alias those lists. Python floats are boxed objects. Dependencies are pyperf and standard Python; the final optimized software uses no external numeric library. All inputs and masses are read from supplied state; no trajectories or answers are precomputed.

Conceptually, for each pair $\delta = r_1 - r_2$ and $\text{mag} = dt * (\delta \cdot \delta) ** (-1.5)$. The two velocity increments are opposite, weighted by the other body's mass. Only after all ten pairs does each position advance by $dt * \text{velocity}$. Energy reporting sums kinetic energy and negative pair potential; momentum offset initializes the reference body before timing. The implementation preserves the reference's exact expression order, not just these real-number identities.

The wrapper loads fresh upstream state for each independent measurement and calls `upstream.bench_nbody`. Momentum offset occurs before its internal timer. Each timed unit runs energy reporting, advance, then energy reporting. The optimized wrapper substitutes only advance; validation, loading and writeback in that function remain timed. Energy reporting and the upstream loop remain unchanged.

Latest pair: `results/nbody_baseline_20260920-031836` and `results/nbody_optimized_20260920-032819`. Comparison: `results/comparison_nbody_20260920-033626`. Kernel `flat_pow` is explicitly recorded; current variant SHA matches the run.

Use independent instruments for different questions

The target is Ubuntu 22.04.5, Linux 5.15.0-1106-kvm, one virtual CPU exposing an Intel Xeon E5-2630 v3 at 2.40 GHz. Clean timing uses release CPython 3.10.12, CPU 0 affinity, a fixed hash seed and cyclic GC disabled. The VM governor is unavailable; host scheduling, frequency and contention are not fully controlled. These are measurements of this guest, not bare-metal Haswell limits.

Instrument	What it establishes	What it cannot establish
Clean timing	11 fresh processes per arm; upstream internal timer; every observation retained.	No proof against between-block drift.
perf stat	Separate repeated counting runs; instructions, cycles, branches and loads per equal work.	Its elapsed time is not the clean timing result.
perf record + FlameGraph	Native debug stacks, fixed 5-million-cycle sampling period; aggregate all folded stacks.	Not a release-time percentage or a count of Python calls.
cProfile	Exact call counts and Python function attribution in a separate run.	Tracing time is perturbed and is not a speedup measurement.
pyperformance / pyperf	Official suite baseline cross-check in its worker harness.	No official optimized-suite result is present.

The custom wrapper calls the actual upstream bench function and uses its timer. A minimal pyperf import stub exposes `time.perf_counter`; it does not implement `pyperf.Runner`. Thus this is an upstream-workload experiment through a custom harness, alongside a separate official pyperformance baseline run. That distinction does not invalidate the experiment; the workload, timer boundaries and comparison controls must be explicit.

Runtime reduction = $100 \times (1 - \text{optimized median} / \text{baseline median})$. Speedup = $\text{baseline median} / \text{optimized median}$. We use runtime reduction for the declared 7% target. CV = $\text{sample standard deviation} / \text{mean}$. Samples from separate TXT and CSV perf invocations are not pooled.

Container overhead was real, but not the whole cost

A flame graph is an aggregation of sampled call stacks, not a timeline. Width is inclusive sampled weight; height is stack depth, not elapsed time. Self weight counts only samples whose leaf is that symbol. Children weight includes descendants and overlaps other ancestors. The libc start routine can therefore have a huge inclusive share while doing little direct work. Aggregate a function across every folded stack: the top-30 listing is incomplete, and repeated appearances must not be mistaken for separate causes.

In the newest baseline native profile, PyObject_GetItem and PyObject_SetItem subtrees cover 226 of 1,174 samples (19.25%). Evaluator self weight is 511 (43.53%). Float operators, binary dispatch, boxing and release are also prominent. These categories can overlap when inclusive; do not sum them into a partition of runtime.

The source explains the stack evidence. Every pair unpacks positions and masses; each `v[i]` update reads and writes a list element through Python's generic numeric/container machinery. A cache hit still needs interpreter instructions, bounds/index handling, reference updates and dispatch. Keeping a float reference in a local avoids those container operations even when the entire physical working set fits in cache.

Source-level work per timestep	Upstream	grouped	flat_pow
Logical component reads + writes	$150 + 75 = 225$	$114 + 57 = 171$	30 initial reads + 30 final writes per advance, outside the timestep loop
Pair traversal	10 entries / step	4 partner groups / step	10 explicit updates / step
Float-result expressions	270	270	270
Power expression	$dt * d2^{**}(-1.5)$	Same	Same

These are source-operation counts, not machine-load counts or runtime percentages. Entry shape validation and mass/container access are excluded from the component count. All float arithmetic remains: 5.4 million float-result expressions per unit. Locals still hold boxed floats, and interpreter freelists can reuse storage; neither an expression count nor a constructor call equals a fresh malloc.

CPython is a bytecode interpreter, not an AST interpreter. `_PyEval_EvalFrameDefault` includes bytecode execution, operand handling, loop control and reference work; its self share is not pure call overhead. Debug checks and different code generation change costs. High IPC and few cache misses support an instruction-work hypothesis but do not identify a precise hardware stall limit. Reject negative top-down fractions and unsupported PMU events rather than interpreting them as valid measurements or zeros.

Why grouped was modest; why flattening is different

grouped retains the first body's position and velocity across its consecutive partners, writes its velocity once per group, and updates the second body immediately. It preserves upstream pair and arithmetic order, including pow. It removes 24% of logical component accesses but still traverses groups, unpacks tuples and reloads state on every timestep. Four partner iterators per step add overhead. Retaining pow was intentional, not an experiment defect.

flat_pow loads all 30 position/velocity components into scalar locals once per advance, retains them through all timesteps, explicitly expands the ten pairs and five position updates in their original order, and writes state back once. It simultaneously removes most state subscripting, repeated unpacking and short-loop traversal. The outer timestep loop, actual physics, power expression and update ordering remain.

The specialization supports five bodies, canonical ordered pairs, distinct ordinary position/velocity lists of floats, float dt and nonnegative integer steps. Validation rejects unsupported shapes. It preserves list identity and successful-call state; delayed writeback does not preserve partial updates on exceptions, signal-time observations or arbitrary aliased/custom containers. This is an explicit benchmark contract, not universal Python API equivalence.

Candidate	What is changed	Status
sqrt	Replace inverse power by $dt / (d2 * \text{sqrt}(d2))$.	Approximate; not final default.
hoist / full	Legacy per-pair hoisting; full also uses sqrt.	Retained controls; full is not flat_pow.
flat_sqrt	All flat_pow structural changes plus sqrt rewrite.	Tolerance passes; bit identity fails.
Structural controls	Unpacked, indexed-grouped, unrolled-pairs/lists, velocity-locals and full-state controls.	Added in experiments/nbody_controls.py; exact checks pass.

A repeated small saving matters over 20,000 steps, but repetition multiplies both old and new work; it does not automatically multiply percentage speedup. The grouped and flat results show that the scope of state reuse, not merely the presence of local variables, determines the impact.

Correctness, alternatives and decisions

Acceptance for grouped and flat_pow requires finite, bit-identical energy and all 30 state components, checked as binary floating-point values. Energy alone is insufficient: different trajectories can share a similar total energy. Tests cover zero steps, small trajectories, randomized valid states/masses, list identity, invalid layout rejection and full accumulated batches.

The 16 x 20,000-step comparison returns energy -0.1690801979358367 and matches all state bits for grouped and flat_pow on the target VM. The newly supplied profiling directory saved an automatic one-unit verification; the separate user-provided 16-unit transcript supplies the longer check. Current scripts have been changed to verify the actual requested timing batch. All 63 integrated software tests pass locally; this is tested agreement, not a proof over every possible input.

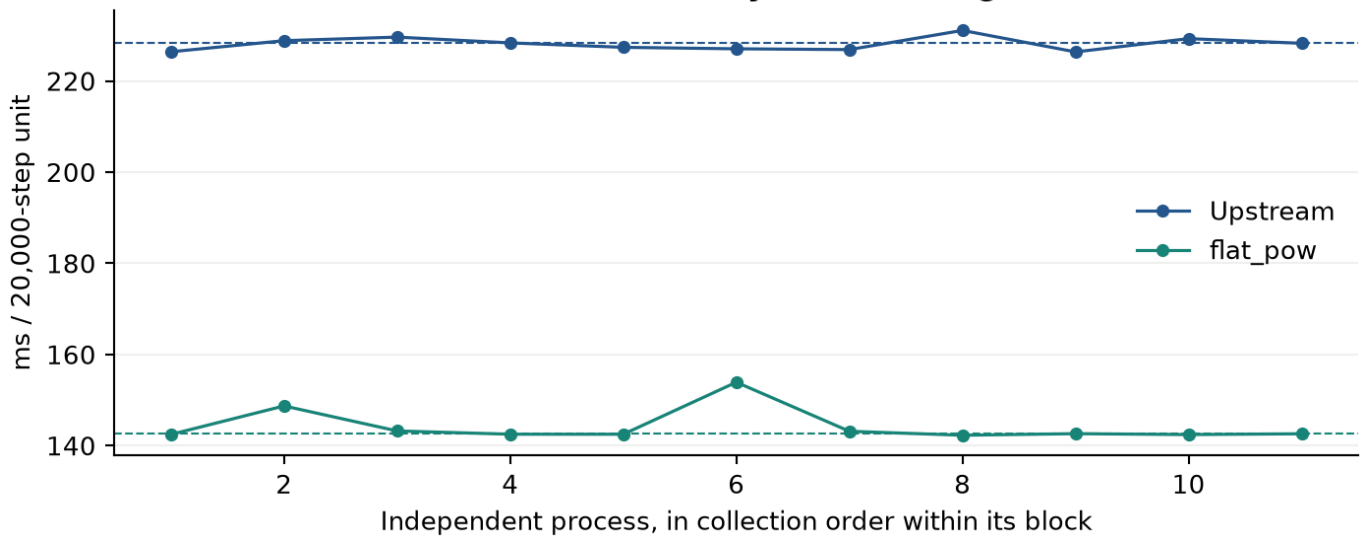
Idea	Reason / evidence	Decision
sqrt rewrite	At 16 units, relative energy error 2.364e-14; max state error 2.522e-10; normalized error 1.004e-11.	Retain as optional, reject as exact default.
Barnes-Hut / FMM	Only five bodies; approximation/tree overhead and different force evaluation.	Reject for this exact workload.
Fewer steps / new integrator	Changes the requested computation and trajectory.	Reject.
Plain NumPy tiny operations	Per-call overhead for five bodies; no measured benefit; reductions may reorder arithmetic.	Not implemented without evidence.
Compiled native backend	Could remove boxed interpretation cost; changes execution technology.	Deferred by agreed pure-Python scope.
More unrolling / expression reassociation	Timestep expansion increases code size; reassociation changes FP rounding.	No unverified addition to default.

The decision is to keep the measured exact flat_pow implementation and add diagnostic controls rather than claim that every proposed rewrite improves it. flat_sqrt contains the same flat/unrolled structure with a different arithmetic choice; there is no missing separate flat_unrolled kernel.

Contracts, commands and previous validation are retained in docs/NBODY_FLAT_EXPERIMENT.md, docs/NBODY_GROUPED_EXPERIMENT.md, docs/NBODY_CONTROL_EXPERIMENTS.md and tests/.

The latest full run clears the runtime goal

Latest full Ubuntu run: every clean timing observation



Newest full run: 11 fresh processes per arm, 16 accumulated units per process. All samples retained. Separate collection blocks leave host drift as a limitation.

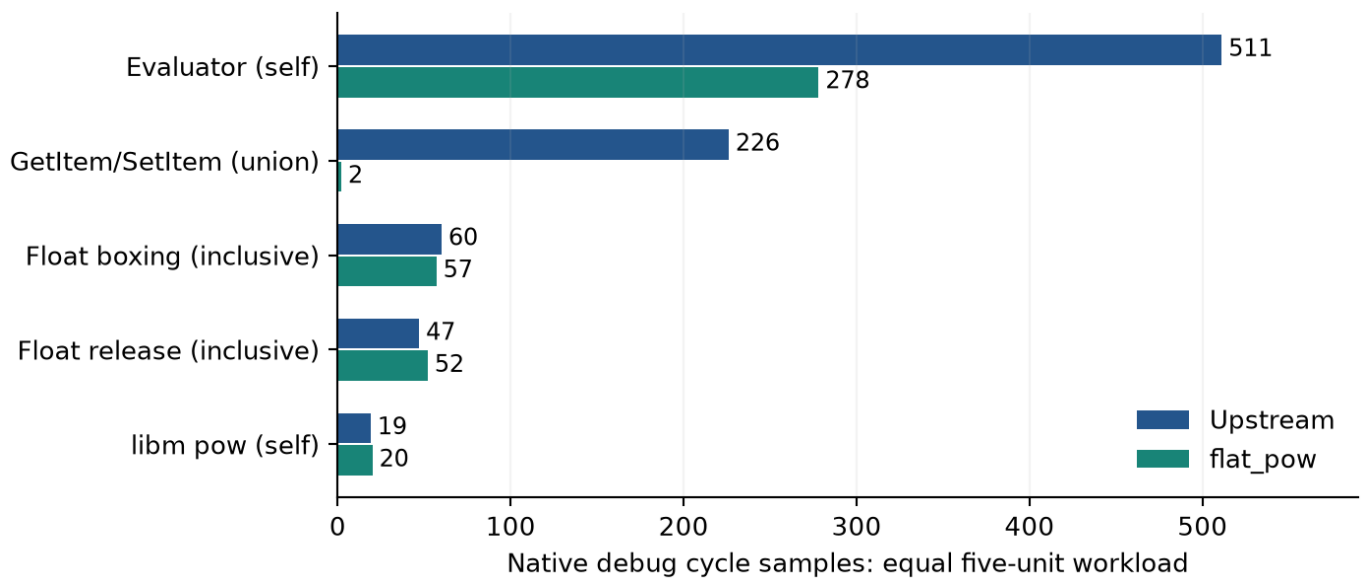
Clean timing	Upstream	flat_pow
Median / 20,000-step unit	228.24925 ms	142.578375 ms
Mean +/- sample SD	228.128 +/- 1.491 ms	144.193 +/- 3.707 ms
CV / range	0.654%; 226.360-231.080 ms	2.571%; 142.253-153.887 ms
Median process batch	3.651988 s	2.281254 s

The median reduction is 37.534% (1.6009x speedup), saving 85.67 ms per unit. A 7% reduction would require at most 212.27 ms. The actual result has substantial margin. Every optimized observation is faster than every baseline observation; dispersion is small relative to the effect. Sequential arm collection still prevents a claim that drift has been fully excluded.

The earlier grouped pair was 227.314 -> 219.589 ms (3.40%). A separate flat_pow timing-only pair gave 37.77%, consistent with this full workflow. The optional flat_sqrt pair gave 38.58%; its 140.702 ms median was only about 1.28% below the earlier flat_pow median, across separate blocks. That small incremental difference is not a controlled direct comparison and does not justify sacrificing bit identity.

Official pyperformance 1.14.0 / pyperf 2.10.0 baseline: about 230 +/- 3 ms, consistent with the custom 228.25 ms baseline. There is no corresponding official optimized-suite run; do not relabel the custom result.

The new profiles isolate the removed work



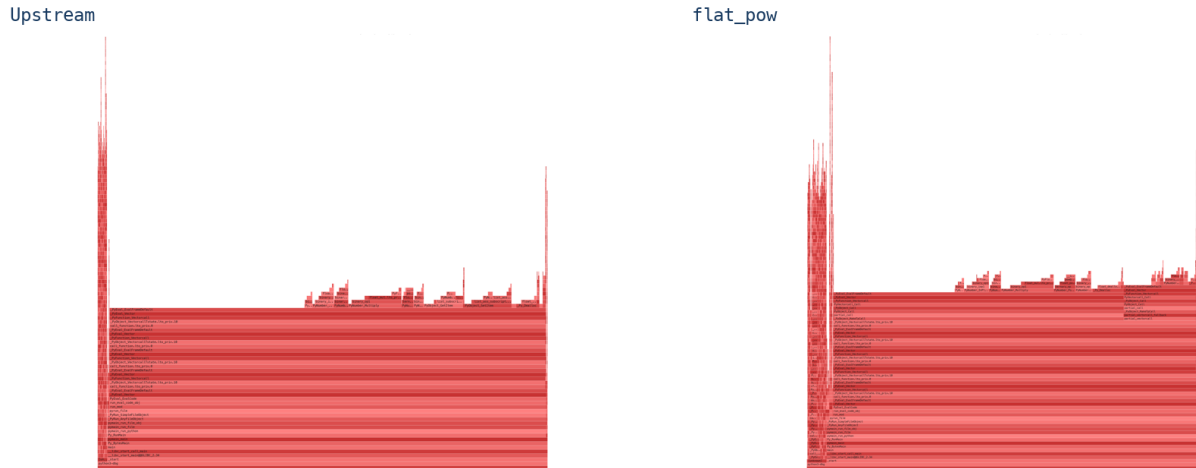
Equal native work: five units per arm, debug CPython, fixed 5-million-cycle sample period. Subscript and boxing/free rows are inclusive unions; evaluator and libm pow rows are self. Do not sum rows.

Counter, per 20,000-step unit	Upstream	flat_pow	Change
Instructions, TXT batch	1.69191 billion	1.10005 billion	-34.98%
Cycles, TXT batch	544.895 million	347.711 million	-36.19%
L1 data loads, TXT batch	535.487 million	351.029 million	-34.45%
Independent CSV batch	Instructions -34.87%	Cycles -36.43%	Loads -34.58%

The largest direct signature is disappearance of generic state indexing: the GetItem/SetItem union falls 226 -> 2 samples, while evaluator self falls 511 -> 278. Total native samples fall 1,174 -> 726. This matches the source mechanism and the instruction/runtime reductions. It is stronger evidence for the optimization than the baseline profile alone.

The new remaining cost is boxed arithmetic and interpreter execution, not repeated list updates. Float creation inclusive samples are 60 -> 57, float release 47 -> 52 and libm pow self 19 -> 20. Those small counts are noisy, but clearly do not show arithmetic disappearing. Their shares rise because other work was removed. A pow sample share is not a hard upper bound for a rewrite that also changes dispatch, boxing and dependencies.

Read the similar stacks without missing the change



Structural overview of the latest original SVGs. Each fills its own panel, hiding the difference in total sample count. Original SVGs alongside this PDF retain interactive zoom.

Both runs still enter the same upstream timed loop, call advance and execute floating-point operations through CPython. Those common stack bases dominate the visual silhouette. The flat wrapper adds a different call path, so depth is not a direct work measure. Look for the missing subscription branches and compare equal-work leaf/subtree counts instead of only the outline.

cProfile calls advance once per unit; nbody is not the raytrace story of millions of small Python method calls. Total process calls actually rise from 4,911 to 25,256, largely due to optimized-wrapper imports/startup and validation. The hot arithmetic remains inside one advance call. Native interpreter helpers are not separate Python source calls. Top-30 lists omit many stacks; full folded aggregation avoids mistaking partial listings for the whole profile.

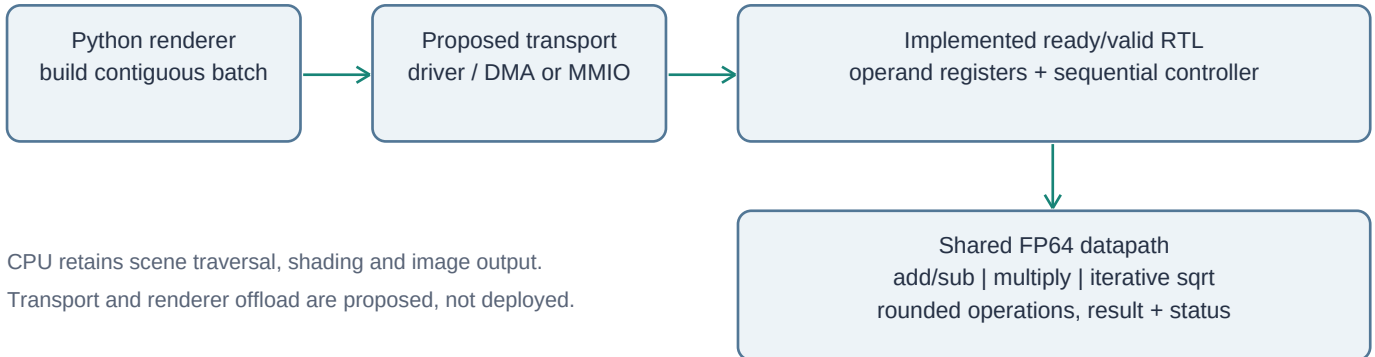
Both independent counter batches support the major instruction, cycle and load reductions. L1 load misses increase 5.48% (TXT) and 9.26% (CSV): this is not a cache-miss improvement. TXT miss rate rises from 0.0504% to 0.0811% while loads fall 34.45%. IPC changes modestly, 3.105 to 3.164. Fewer executed instructions dominate. PMU coverage is roughly 40-60%; generic cache misses are noisy, LLC is unsupported, and negative top-down bad-speculation fractions invalidate that decomposition.

The logged variant hash matches the current flat_pow source. Both arms record upstream SHA d1385e816d7cfea361b7915e2cf70138cd6b84f40df8bd5152638851f7bcac2b. However, manifests record 13/14 dirty files, an UNVERIFIED provenance placeholder and no full authenticity-gate transcript. Hash identity establishes the local source, not independently its published origin. The separate official baseline supports workload scale; retain future package-hash comparisons and complete manifests for stronger provenance.

The remaining bottleneck is a set of costs: arithmetic dispatch, boxed values, evaluator work and pow. An exact next software improvement must be tested against flat_pow. The new structural controls can distinguish locality from unrolling; the current aggregate speedup cannot uniquely allocate savings among them.

A small, explicit FP64 accelerator

One ray-sphere accelerator is implemented for the project. It is relevant to raytrace; a separate exact nbody engine is deferred.



Implemented accelerator boundary is the ready/valid RTL. The host transport is an integration proposal.

hardware/rtl/ray_sphere_accel.sv contains the sequential controller, operand/result registers, status and ready/valid protocol. The datapath uses actual unmodified BSD-licensed Berkeley HardFloat Release 1 RTL: one shared add/sub unit, one multiplier and an iterative square-root unit. There are no real-number or software-math placeholders in the synthesizable datapath.

```

cp = center - origin
v = (cp.x*dx + cp.y*dy) + cp.z*dz
q = (cp.x*cp.x + cp.y*cp.y) + cp.z*cp.z
disc = r*r - (q - v*v)
result = no_root if disc < 0 else v - sqrt(disc)
  
```

Each operation rounds separately to binary64, nearest-even, including gradual underflow. There is no FMA, reassociation, FP32 or approximate reciprocal root. A tangent and a negative returned distance preserve upstream behavior; software applies its original hit-selection rule. Finite inputs/intermediates, nonnegative radius and caller-normalized direction define the supported domain. Invalid/nonfinite cases return an error for CPU fallback.

HardFloat source and license: hardware/vendor/ provenance and license files. Primary specification: <https://www.jhauser.us/arithmetic/HardFloat-1/doc/HardFloat-Verilog.html>. Detailed ports, schedule and host format: docs/HARDWARE_DESIGN.md and hardware/.

Interfaces, software stack and honest estimates

Item	Specification / status
Request	640 operand bits (10 FP64 values) plus 32-bit tag; ready/valid; one outstanding query.
Response	64-bit distance, tag, root/error status and 5 FP exception flags; stable under backpressure.
Control	Reset discards in-flight work; explicit sequential FSM; completion returns to accepting state.
Software	Host packing/reference helper exists. Batched renderer scheduling, driver and transport/DMA are proposed, not deployed.
Frequency / schedule	50 MHz target assumption; 20-cycle miss / 80-cycle root initiation budgets. Not synthesis or simulation results.
Validation	Syntax/elaboration and host-format checks only. No HDL behavioral simulation, timing analysis or physical measurement.

Illustrative pessimistic case: 179,457 queries/frame, all taking a root; 50 MHz; 1 GB/s transport; batches of 1,024 with 10 us launch cost; 100 ns packing/query. The model gives 287.13 ms computation + 18.66 ms transfer + 1.76 ms launch + 17.95 ms packing = 325.50 ms offload cost. These are assumptions, not measured hardware performance.

Use $T_{\text{new}} = T_{\text{software}} \times (1 - f) + T_{\text{offload}}$, where f is the release-time fraction actually removed by a real batch implementation. Starting from the measured 611.02 ms shadow-ray software, break-even needs $f > 53.27\%$. With $f = 50\%$, the estimate is 631.01 ms: 3.27% slower. With $f = 70\%$, it is 508.81 ms: 16.73% lower time. Neither f is established by the debug profile, and the newer full software requires a new baseline.

A single shared datapath limits area and switching relative to many parallel lanes, but limits throughput; FP64 sqrt/rounding still costs substantial logic. Deeper pipelining or replication trades area, clocking and power for throughput. No LUT, gate, watt or achieved-clock numbers are claimed. The proposal needs batch amortization and must eventually be compared with an equivalent compiled CPU batch, since bypassing Python alone may explain most gains.

The assignment requires a complete logical HDL design and interface/architecture estimates (pp. 4-5), not physical implementation. Source completeness and syntax checks do not prove functional correctness. Hardware numerical/pixel acceptance and real transport integration remain future work.

A successful exact optimization, with bounded claims

The newest measured flat_pow result exceeds the declared 7% runtime goal by a wide margin while retaining the power expression and exact accumulated state. The grouped experiment was useful: it showed that reducing some list accesses was insufficient. Full-state locality and explicit pair/position updates remove substantially more repeated interpreter work, as the new native stacks and counters confirm.

A five-body hardware engine could hold state locally and execute complete advance batches, amortizing transfer across 20,000 steps. It is not the implemented accelerator: exact agreement with the host nonintegral pow routine, dependencies between velocity updates and full trajectory validation make it a different, larger design. A MAC/rsqrt replacement would change rounding and is unsuitable for the strict final contract without further proof and testing.

One concrete ray-sphere HDL design satisfies the chosen single-component hardware scope. Its architecture and losing as well as winning estimate scenarios are documented above. It does not speed up nbody, and syntax checks are not a claim of numerical correctness or achieved acceleration.

```
./run_all.sh
python3 -B variants/bm_nbody_upstream_opt.py \
--mode verify --loops 16 --iterations 20000
LOOPS=16 ./tools/ab_timing.sh nbody 16 --kernel flat_pow
python3 -B experiments/nbody_controls.py --mode verify \
--loops 16 --iterations 20000
```

Required deliverable	Location / evidence
Six-section TXT report	report_nbody.txt; this PDF is supplemental and uses the same source.
Execution scripts / README	script_nbody.sh, run_all.sh, tools/, README.md.
Variants and exact checks	variants/bm_nbody_upstream_opt.py; tests/; docs/evidence/.
Latest full evidence	results/031836 baseline, 032819 optimized, 033626 comparison; full names on page 1.
Analysis / hardware / AI disclosure	docs/reports/, hardware/, docs/HARDWARE_DESIGN.md, prompt.txt.
Remaining submission actions	Repository upload, 20-25-minute presentation and working demo; new full raytrace VM measurement.

Project.pdf pp. 3-5 and 7-8 drive these sections; docs/PROJECT_REQUIREMENTS.md maps each requirement. No Git operations were performed. New measurements require deliberate source/report updates; the pipeline will not overwrite authored interpretation.